COMPTE RENDU — TP 4

Reseaux de Neurones Convolutifs — Optimisation et Regularisation

PyTorch — CIFAR-10 | Annee universitaire 2025-2026

Partie 1 – Questions theoriques sur les CNNs

1. Taille de sortie d'une convolution et nombre de poids ?

Pour un filtre de taille k, stride s, padding p sur une entree x x y x z :

Taille de sortie = floor((x - k + 2p)/s + 1) x floor((y - k + 2p)/s + 1) x 1

Nombre de poids du filtre = k x k x z + 1 (biais).

Une couche fully-connected produisant la meme sortie aurait besoin de :

(x x y x z) x (sortie_x x sortie_y) poids — soit enormement plus.

2. Avantages de la convolution vs fully-connected ? Quelle est sa limite principale ?

Avantages :

- Partage de poids : un meme filtre est applique a toute l'image → bien moins de parametres.
- Invariance locale par translation : un motif detecte a un endroit l'est partout.
- Exploite la structure spatiale 2D des images.

Limite principale : la convolution n'est pas invariante aux grandes transformations geometriques (rotation, echelle). Elle capte uniquement des dependances locales definies par la taille du kernel.

3. Quel interet voyez-vous a l'usage du pooling spatial ?

- Reduit les dimensions spatiales → moins de calculs et de parametres dans les couches suivantes.
- Apporte une invariance locale aux petites translations (le max-pooling garde l'activation la plus forte dans une fenetre).
- Augmente le champ recepteur effectif des couches profondes.

4. Peut-on calculer un CNN classique sur une image plus grande que prevu ?

Non directement, car les couches fully-connected attendent un vecteur de taille fixe. Si l'image est plus grande, les feature maps apres les convolutions/poolings seront plus grandes, et le Flatten produira un vecteur de taille differente → erreur de dimension dans la premiere couche FC.

5. Montrer que les couches fully-connected peuvent etre vues comme des convolutions particulieres.

Une couche FC qui prend un vecteur de taille H x W x C et produit N sorties est equivalente a une convolution avec N filtres de taille H x W x C et stride 1, sans padding. Le kernel couvre exactement toute la feature map d'entree (champ recepteur = taille entree), ce qui est mathematiquement identique au produit matrice-vecteur de la couche FC.

6. Si on remplace les FC par des convolutions, que peut-on faire avec des images plus grandes ?

En remplacant les FC par des convolutions equivalentes, le reseau devient entierement convolutionnel (FCN). On peut alors traiter des images de n'importe quelle taille. La sortie n'est plus un vecteur de taille N mais une carte spatiale de taille H' x W' x N, ou chaque position (i,j) correspond a la prediction pour la region correspondante de l'image d'entree. Cela permet la detection ou la segmentation denses en un seul passage.

7. Tailles des champs recepteurs (receptive fields) des neurones ?

Architecture consideree (conv 5x5, stride 1, padding 2 / pool 2x2, stride 2) :

- Apres conv1 (5x5) : receptive field = 5x5 pixels.
- Apres pool1 (2x2) : receptive field = 6x6 pixels (le pooling agrandit le champ).
- Apres conv2 (5x5 sur la sortie du pool) : receptive field ≈ 14x14 pixels.

Plus on va profond, plus le champ recepteur est grand : les neurones profonds 'voient' une plus grande region de l'image et capturent des caracteristiques de haut niveau.

Partie 2 – Apprentissage from scratch

2.1 Architecture du reseau (style AlexNet adapte CIFAR-10)

8. Valeurs de padding et stride pour conserver les dimensions spatiales en sortie de conv ?

Pour un kernel 5x5 et conserver la taille spatiale : padding = 2, stride = 1.
Verification : floor((32 - 5 + 2x2)/1 + 1) = 32. La taille est conservee.

9. Valeurs de padding et stride pour le max-pooling 2x2 reduisant d'un facteur 2 ?

kernel_size = 2, stride = 2, padding = 0.
Verification : floor((32 - 2)/2 + 1) = 16. Taille reduite de moitie.

10. Taille de sortie et nombre de poids par couche ?

```
Entree  :  32 x 32 x 3
conv1   :  32 x 32 x 32  | poids = (5x5x3+1)x32  =   2 432
pool1   :  16 x 16 x 32  | poids = 0
conv2   :  16 x 16 x 64  | poids = (5x5x32+1)x64 =  51 264
pool2   :   8 x  8 x 64  | poids = 0
conv3   :   8 x  8 x 64  | poids = (5x5x64+1)x64 = 102 464
pool3   :   4 x  4 x 64  | poids = 0
fc4     :        1000    | poids = (4x4x64+1)x1000 = 1 025 000
fc5     :          10    | poids = (1000+1)x10      =    10 010
```

Observation : fc4 concentre l'immense majorite des poids (~85% du total). Les couches de convolution extraient les caracteristiques avec peu de poids grace au partage, alors que la couche FC est tres couteuse en parametres.

11. Nombre total de poids ? Comparaison avec le nombre d'exemples.

Total ≈ 2 432 + 51 264 + 102 464 + 1 025 000 + 10 010 = 1 191 170 parametres.

CIFAR-10 contient 50 000 exemples d'apprentissage. Le modele a donc environ 24x plus de parametres que d'exemples → risque eleve de sur-apprentissage sans techniques de regularisation.

12. Comparaison du nombre de parametres avec l'approche BoW + SVM.

Une approche BoW (Bag of Words) avec descripteurs SIFT et un SVM lineaire necessite generalement quelques dizaines de milliers de parametres (taille du dictionnaire visuel x nombre de classes). Le CNN ici a ~1.2M de parametres, soit 10 a 100x plus. En contrepartie, le CNN apprend ses propres representations et surpasse largement le BoW+SVM sur CIFAR-10 (~75-80% vs ~50-60%).

2.2 Apprentissage du reseau

13. Lire et tester le code fourni (LeNet-5 sur MNIST).

Le code (base.py / tp_4_cnn_optimisation.py) definit un ConvNet de type LeNet-5 entraine sur MNIST avec PyTorch. La fonction main() lance l'entrainement :

```
main(batch_size=128, lr=0.1, epochs=5, cuda=True)
```

La boucle principale alterne une phase train (avec optimizer) et une phase eval (sans optimizer) a chaque epoch, en affichant loss et accuracy.

14. Difference importante entre le calcul de la loss/accuracy en train et en test ?

En train : optimizer.zero_grad(), loss.backward() et optimizer.step() sont appeles pour mettre a jour les poids. Le modele est en mode model.train().

En test : aucune retropropagation n'est effectuee (pas d'optimizer passe a epoch()). Le modele est en mode model.eval(), ce qui desactive des couches comme Dropout et ajuste le comportement de BatchNorm. Les gradients ne sont pas calcules, ce qui economise de la memoire et du calcul.

15. Modifier le code pour CIFAR-10 avec l'architecture demandee. Resultats ?

On remplace datasets.MNIST par datasets.CIFAR10 et on adapte ConvNet avec les couches conv1-pool1-conv2-pool2-conv3-pool3-fc4-fc5 decrites en 2.1. L'entree passe de (1,28,28) a (3,32,32).

Resultats obtenus apres convergence (~20-30 epochs, lr=0.01) :
- Accuracy train : ~75-80%
- Accuracy test : ~65-70%

L'ecart train/test indique un debut de sur-apprentissage.

16. Effets du learning rate et de la taille de mini-batch ?

Learning rate :
- lr trop grand (ex. 0.1) : divergence ou oscillations de la loss.
- lr trop petit (ex. 0.0001) : convergence tres lente.
- lr optimal (~0.01 avec SGD) : convergence stable et rapide.

Taille de mini-batch :
- Petit batch (ex. 32) : gradient bruité mais meilleure generalisation, epochs plus lentes car plus de steps.
- Grand batch (ex. 256) : gradient lisse, convergence rapide par epoch mais risque de minima moins bons.

17. A quoi correspond l'erreur au debut de la premiere epoque ?

Au debut, les poids sont initialises aleatoirement. Le reseau predit donc de facon quasi-aleatoire parmi les 10 classes.
Loss initiale attendue = -log(1/10) = log(10) ≈ 2.30 (cross-entropie).
Accuracy initiale ≈ 10% (chance pure).
Cela confirme que le modele part d'une prediction aleatoire et que l'apprentissage fonctionne bien des qu'on observe la loss diminuer.

18. Interpreter les resultats. Qu'est-ce qui ne va pas ?

On observe un ecart croissant entre la loss d'entrainement (qui diminue) et la loss de test (qui stagne ou remonte). C'est le phenomene de sur-apprentissage (overfitting) : le modele memorise les exemples d'entrainement sans generaliser aux nouvelles images. Cela s'explique par le grand nombre de parametres (~1.2M) par rapport aux 50k exemples et l'absence de regularisation.

Partie 3 – Ameliorations des resultats

3.1 Normalisation des exemples

19. Decrire vos resultats experimentaux.

Apres ajout de la normalisation par channel RGB (mu=[0.491,0.482,0.447], sigma=[0.202,0.199,0.201]) via transforms.Normalize() :

- La loss initiale diminue plus vite (meilleur conditionnement du gradient).
- L'accuracy de test augmente de ~2-3 points par rapport a la baseline.
- La convergence est plus stable, avec moins d'oscillations.

20. Pourquoi ne calculer la moyenne que sur les exemples d'apprentissage ?

Les statistiques de normalisation (moyenne et ecart-type) doivent etre calculees uniquement sur le train set pour eviter toute fuite d'information (data leakage) des donnees de validation/test vers le modele. En pratique, on applique ensuite ces memes valeurs pour normaliser les exemples de validation et de test, simulant ainsi une situation reelle ou les statistiques de la population sont estimees uniquement a partir des donnees disponibles a l'entrainement.

3.2 Augmentation du nombre d'exemples (Data Augmentation)

22. Decrire vos resultats et les comparer aux precedents.

Avec crop aleatoire 28x28 et flip horizontal aleatoire en train (+ crop centre 28x28 en test) :

- L'accuracy de test augmente de ~3-5 points supplementaires.
- L'ecart train/test se reduit : le sur-apprentissage diminue car le modele voit plus de variantes de chaque image.
- La convergence est plus lente (les exemples sont plus difficiles) mais le modele final generalise mieux.

23. La symetrie horizontale vous semble-t-elle utilisable sur tout type d'images ?

Non. Elle est valide uniquement quand la symetrie n'affecte pas la semantique :

- Valide : photos naturelles (animaux, paysages), visages de face.
- Non valide : texte et chiffres ('b' et 'd' sont differents), panneaux de signalisation, images medicales avec lateralite, images de mains gauche/droite.

24. Quelles limites voyez-vous a ce type de data augmentation ?

- Les transformations restent dans le meme domaine visuel : on ne cree pas vraiment de nouvelles informations.
- Risque de generer des exemples irrealistes si les transformations sont trop agressives (crop trop petit, rotation trop forte).
- Cout calcul supplementaire a chaque epoch.
- Ne remplace pas l'acquisition de vraies nouvelles donnees variees.

3.3 Variantes sur l'algorithme d'optimisation (LR Decay)

26. Decrire vos resultats et les comparer aux precedents.

Avec un ExponentialLR scheduler (gamma=0.95) :

- La loss de validation est plus basse et plus stable en fin d'entrainement.

- Moins d'oscillations dans les dernieres epochs par rapport a un lr fixe.
- L'accuracy de test gagne ~1-2 points supplementaires.
- Le lr diminue de 5% par epoch : lr_t = lr_0 x 0.95^t.

27. Pourquoi cette methode ameliore-t-elle l'apprentissage ?

Un lr eleve en debut d'entrainement permet d'explorer largement l'espace des poids et de converger rapidement vers un bon minimum. En fin d'entrainement, un lr reduit permet d'affiner la solution avec des petits pas, evitant les oscillations autour du minimum. Le reseau 'explore' au debut et 'exploite' (affine) a la fin.

3.4 Regularisation par Dropout

29. Decrire vos resultats et les comparer aux precedents.

Avec une couche Dropout (p=0.5) entre fc4 et fc5 :
- L'ecart entre accuracy train et test se reduit significativement.
- L'accuracy de test augmente de ~2-4 points.
- La loss de validation ne remonte plus aussi vite (moins de sur-apprentissage).

30. Qu'est-ce que la regularisation de maniere generale ?

La regularisation designe l'ensemble des techniques visant a reduire le sur-apprentissage en contraignant la complexite du modele. L'objectif est d'ameliorer la generalisation en penalisant les solutions trop complexes (L2/L1) ou en introduisant de la stochasticite pendant l'entrainement (Dropout).

31. Interpretations possibles de l'effet du Dropout ?

- Interpretation par ensemble : a chaque step, un sous-reseau different est entraine. L'inference finale est une moyenne de ~2^N sous-reseaux.
- Interpretation par bruit : le dropout injecte du bruit dans les activations, forcant le reseau a apprendre des representations robustes et independantes.
- Reduction des co-adaptations : les neurones ne peuvent pas se 'specialiser' en co-dependances fragiles car leurs partenaires sont aleatoirement absents.

32. Quelle est l'influence de l'hyperparametre p du Dropout ?

p est la probabilite d'annuler un neurone :
- p trop faible (ex. 0.1) : regularisation insuffisante, peu d'effet.
- p trop eleve (ex. 0.8) : trop de neurones desactives, le reseau apprend mal (underfitting).
- p=0.5 est la valeur classique pour les couches FC (Srivastava et al. 2014).
- Pour les couches conv, on utilise generalement p plus faible (0.1-0.3).

33. Difference de comportement du Dropout entre apprentissage et evaluation ?

- Apprentissage (model.train()) : chaque neurone est annule avec probabilite p de facon independante a chaque forward pass.
- Evaluation (model.eval()) : tous les neurones sont actifs. Pour compenser, les sorties sont multipliees par (1-p) pour maintenir l'esperance des activations constante (inverted dropout). PyTorch gere cela automatiquement via model.eval().

3.5 Batch Normalization

34. Decrire vos resultats et les comparer aux precedents.

Avec une couche BatchNorm2D apres chaque couche de convolution :

- Convergence nettement plus rapide (moins d'epochs necessaires).
- Possible d'utiliser un lr plus eleve sans divergence.
- L'accuracy de test augmente de ~3-5 points supplementaires.
- La BatchNorm normalise les activations de chaque couche (moyenne≈0, variance≈1) a chaque mini-batch, stabilisant la distribution des entrees des couches suivantes (reduction du internal covariate shift).